LIN7076 – Foundations of Computational Linguistics

Large Language Models

Adèle Hénot-Mortier

09/12/2025

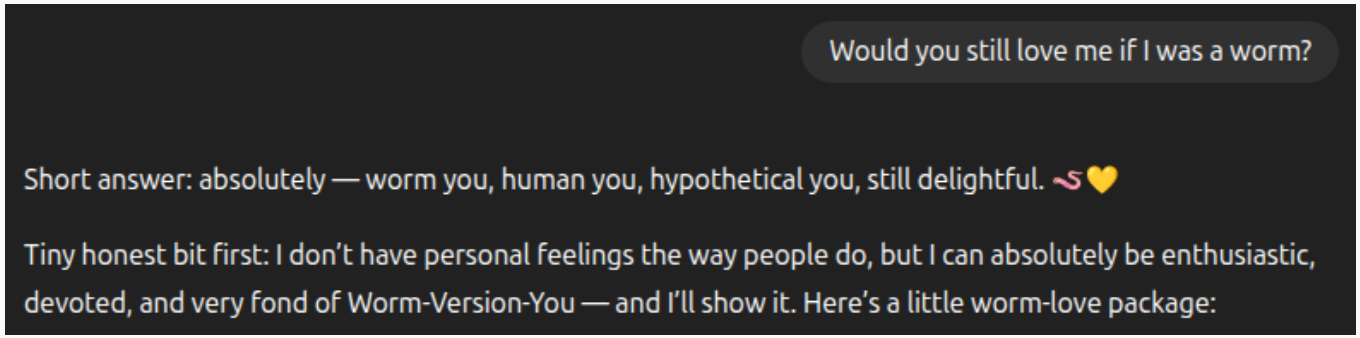
Queen Mary University of London

Plan for today

- Review what Language Models are.
- Introduce the general architecture of Large Language Models.
- Show how LLMs are trained and tuned.
- Show how they can be optimized post-training via prompting.
- Outline current issues with LLMs.

Language Models

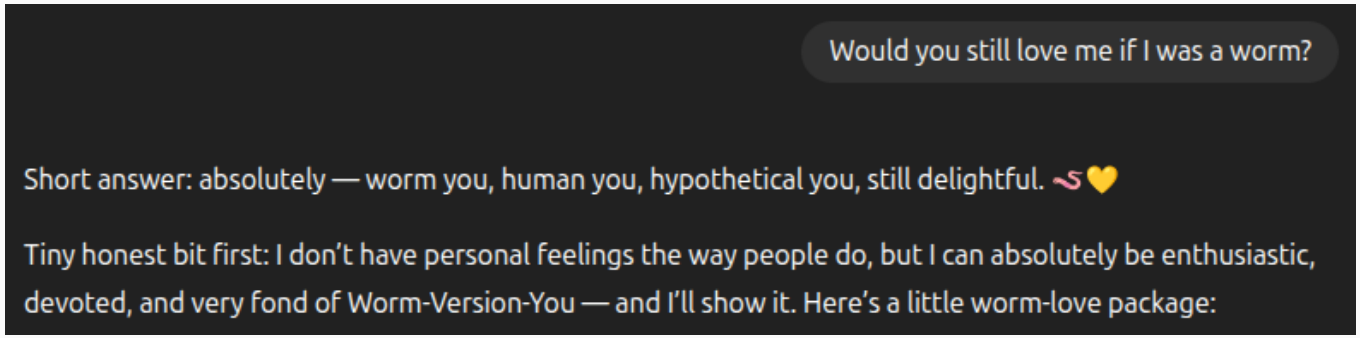
ChatGPT



- Responds to **prompts** by iteratively generating text, token by token.[1]
- Very **versatile**! Can summarize, correct, stylistically improve existing text. Can generate "brand new" pieces of text in different styles (academic, poem, novel...). Can "reason" and produce math proofs. Can answer questions about real-world facts, whether scientific or historical...
- However, it is never 100% accurate, and is known to "**hallucinate**", meaning, it sometimes confidently generates inaccurate answers. Why?

[1] We'll see what tokens are in a bit–think of them as words!

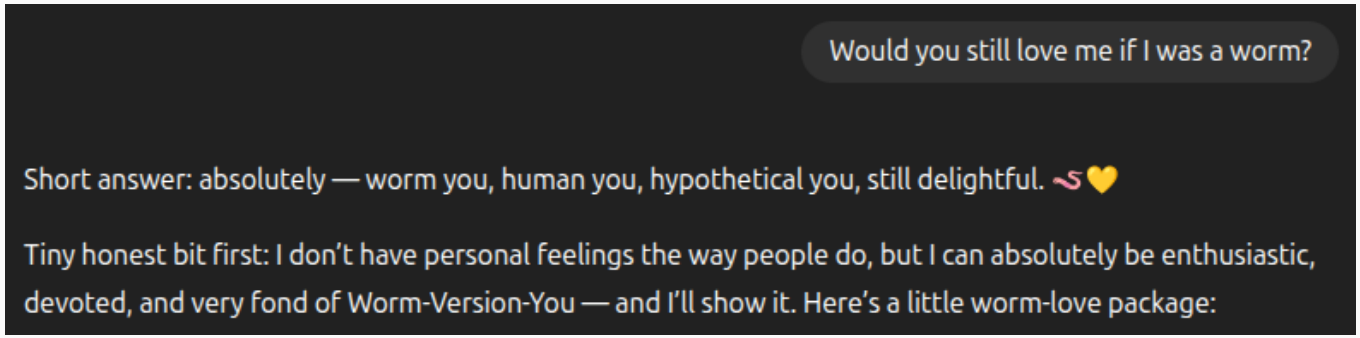
ChatGPT



- Responds to **prompts** by iteratively generating text, token by token.[1]
- Very **versatile**! Can summarize, correct, stylistically improve existing text. Can generate "brand new" pieces of text in different styles (academic, poem, novel...). Can "reason" and produce math proofs. Can answer questions about real-world facts, whether scientific or historical...
- However, it is never 100% accurate, and is known to "**hallucinate**", meaning, it sometimes confidently generates inaccurate answers. Why?

[1] We'll see what tokens are in a bit–think of them as words!

ChatGPT



- Responds to **prompts** by iteratively generating text, token by token.[1]
- Very **versatile**! Can summarize, correct, stylistically improve existing text. Can generate "brand new" pieces of text in different styles (academic, poem, novel...). Can "reason" and produce math proofs. Can answer questions about real-world facts, whether scientific or historical...
- However, it is never 100% accurate, and is known to "**hallucinate**", meaning, it sometimes confidently generates inaccurate answers. Why?

[1] We'll see what tokens are in a bit–think of them as words!

A predecessor: ELIZA

```
  Eliza is a mock Rogerian psychotherapist.
  The original program was described by Joseph Weizenbaum in 1966.
  This implementation by Norbert Landsteiner 2005.


ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

- ELIZA is an early **rule-based** “chatbot” developed from 1964 to 1967 by Joseph Weizenbaum.
- One of the scripts was made for “person-centered” therapy: it was **matching patterns** in the user’s prompts and incorporating these patterns into **templates** asking the user to elaborate.
- Despite the simplicity of the system, some users became emotionally attached to the bot, treating it like a human therapist. You can try different reimplementations here, or here.

A predecessor: ELIZA

```
  Eliza is a mock Rogerian psychotherapist.
  The original program was described by Joseph Weizenbaum in 1966.
  This implementation by Norbert Landsteiner 2005.


ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

- ELIZA is an early **rule-based** “chatbot” developed from 1964 to 1967 by Joseph Weizenbaum.
- One of the scripts was made for “person-centered” therapy: it was **matching patterns** in the user’s prompts and incorporating these patterns into **templates** asking the user to elaborate.
- Despite the simplicity of the system, some users became emotionally attached to the bot, treating it like a human therapist. You can try different reimplementations here, or here.

A predecessor: ELIZA

```
  Eliza is a mock Rogerian psychotherapist.
  The original program was described by Joseph Weizenbaum in 1966.
  This implementation by Norbert Landsteiner 2005.


ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

- ELIZA is an early **rule-based** “chatbot” developed from 1964 to 1967 by Joseph Weizenbaum.
- One of the scripts was made for “person-centered” therapy: it was **matching patterns** in the user’s prompts and incorporating these patterns into **templates** asking the user to elaborate.
- Despite the simplicity of the system, some users became emotionally attached to the bot, treating it like a human therapist. You can try different reimplementations here, or here.

Language Models, general definition

- Unlike ELIZA, which comes with a set of fixed, task-specific, semantics-insensitive pattern-matching and pattern-inserting rules, ChatGPT generates text flexibly and iteratively based on whatever it's given, in a context-sensitive way that incorporates some semantic knowledge.[2]
- A **Language Model** is a system which, given a sequence of tokens, can **predict the next token**. ChatGPT is a very elaborate Language Model; ELIZA is not.[3]

[2] There are many enthusiastic online discussions about using ChatGPT for therapy, how to optimize the prompts etc. Not sure it's an amazing idea, but it's at least interesting that, 60 years after ELIZA, the history repeats in this way!

[3] We'll see what tokens are in a bit–think of them as words!

Language Models, general definition

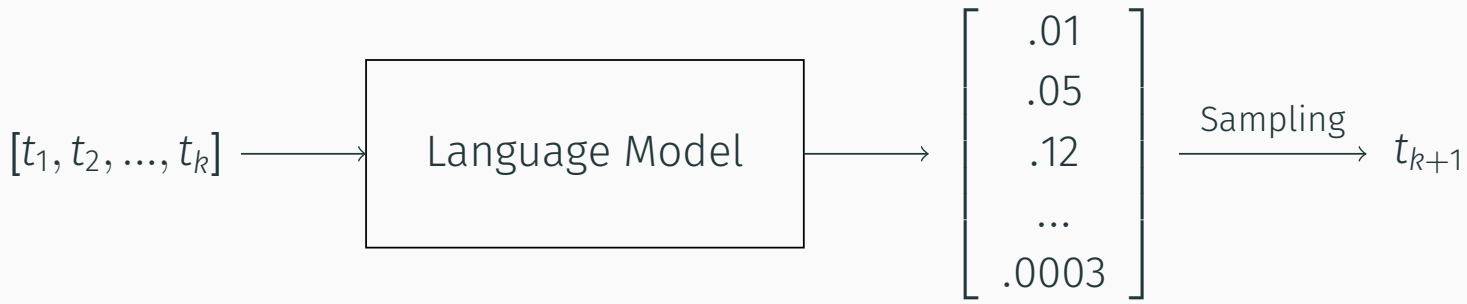
- Unlike ELIZA, which comes with a set of fixed, task-specific, semantics-insensitive pattern-matching and pattern-inserting rules, ChatGPT generates text flexibly and iteratively based on whatever it's given, in a context-sensitive way that incorporates some semantic knowledge.[2]
- A **Language Model** is a system which, given a sequence of tokens, can **predict the next token**. ChatGPT is a very elaborate Language Model; ELIZA is not.[3]

[2] There are many enthusiastic online discussions about using ChatGPT for therapy, how to optimize the prompts etc. Not sure it's an amazing idea, but it's at least interesting that, 60 years after ELIZA, the history repeats in this way!

[3] We'll see what tokens are in a bit–think of them as words!

Language Model, more formally

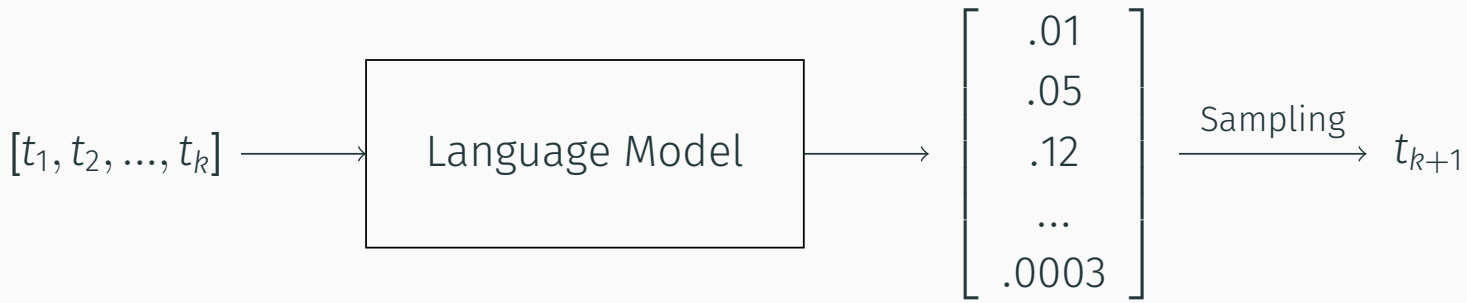
- More formally, a Language Model is fed a representation of the input sequence, and outputs a **probability distribution over possible continuations**.
- This probability distribution takes the form of a **vector of probabilities**, each component corresponding to the probability of a possible continuation.



- The next token is generated by **sampling** over that distribution. This entire pipeline, is called **Language Modeling**.
- n-gram models, RNNs, LSTMs, feedforward neural nets, can act as language models!

Language Model, more formally

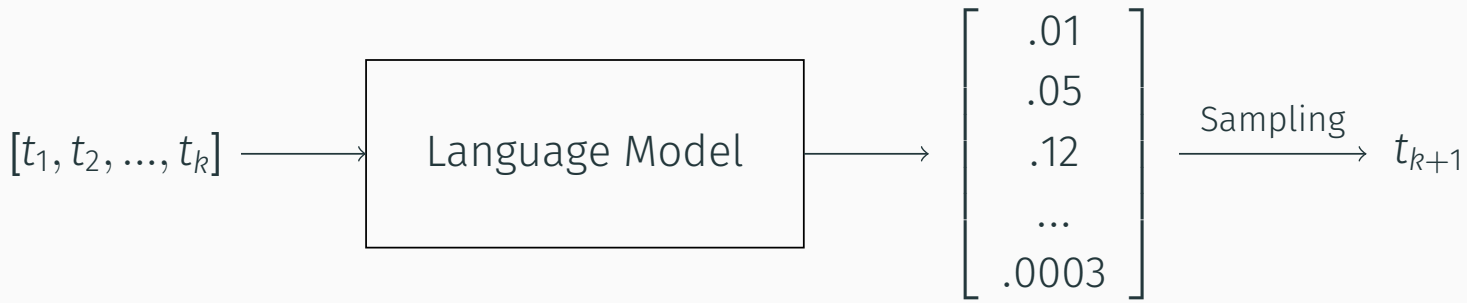
- More formally, a Language Model is fed a representation of the input sequence, and outputs a **probability distribution over possible continuations**.
- This probability distribution takes the form of a **vector of probabilities**, each component corresponding to the probability of a possible continuation.



- The next token is generated by **sampling** over that distribution. This entire pipeline, is called **Language Modeling**.
- *n*-gram models, RNNs, LSTMs, feedforward neural nets, can act as language models!

Language Model, more formally

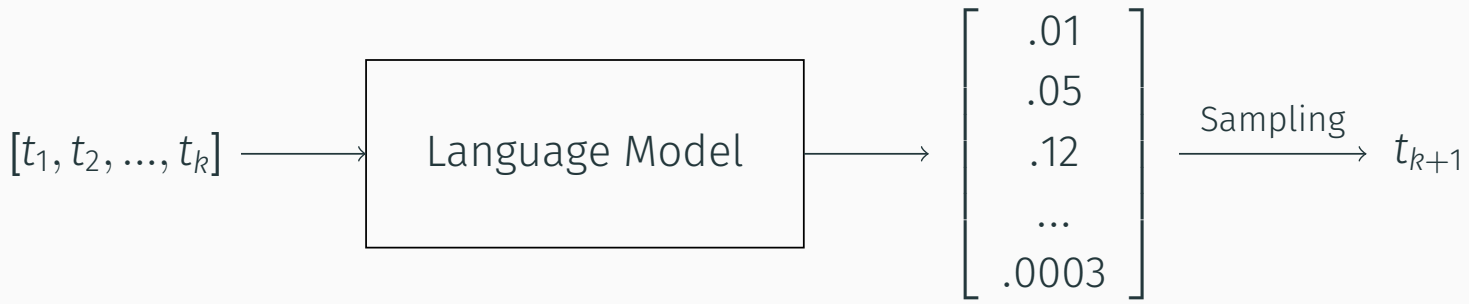
- More formally, a Language Model is fed a representation of the input sequence, and outputs a **probability distribution over possible continuations**.
- This probability distribution takes the form of a **vector of probabilities**, each component corresponding to the probability of a possible continuation.



- The next token is generated by **sampling** over that distribution. This entire pipeline, is called **Language Modeling**.
- *n*-gram models, RNNs, LSTMs, feedforward neural nets, can act as language models!

Language Model, more formally

- More formally, a Language Model is fed a representation of the input sequence, and outputs a **probability distribution over possible continuations**.
- This probability distribution takes the form of a **vector of probabilities**, each component corresponding to the probability of a possible continuation.



- The next token is generated by **sampling** over that distribution. This entire pipeline, is called **Language Modeling**.
- *n*-gram models, RNNs, LSTMs, feedforward neural nets, can act as language models!

n-gram models as Language Models

- A n-gram model is fed a sequence of $n-1$ words $[w_{k-n+2}, ..., w_k]$, and, for any continuation w, can compute the conditional probability:

$$P(w|[w_{k-n+2}, ..., w_k]) \simeq \frac{C([w_{k-n+2}, ..., w_k, w])}{C([w_{k-n+2}, ..., w_k])}$$

- The **vector** $[P(w|[w_{i-n+1}, ..., w_i]) \mid w \in L]$ packaging all the conditional probabilities $P(w|[w_{k-n+2}, ..., w_k])$ for any w, **is a probability distribution over the possible continuations!**

n-gram models as Language Models

- A n-gram model is fed a sequence of $n-1$ words $[w_{k-n+2}, ..., w_k]$, and, for any continuation w, can compute the conditional probability:

$$P(w|[w_{k-n+2}, ..., w_k]) \simeq \frac{C([w_{k-n+2}, ..., w_k, w])}{C([w_{k-n+2}, ..., w_k])}$$

- The **vector** $[P(w|[w_{i-n+1}, ..., w_i]) \mid w \in L]$ packaging all the conditional probabilities $P(w|[w_{k-n+2}, ..., w_k])$ for any w, **is a probability distribution over the possible continuations!**

Feedforward Neural Nets as Language Models

- Pretty much like *n*-grams, feedforward NNs can be **fed with a vector representation of a sequence of words**,[4] and **return a probability distribution over the lexicon**.
- To get a probability distribution in the output, the last layer must have **as many units as the number of words in the lexicon**, and must come with a **softmax** activation.
- The softmax ensures that all the components of the last layer are mapped to $[0; 1]$, s.t. the low values are close to 0, the high values close to 1, and s.t. the components together sum to 1!
- Feedforward NNs are **a bit inefficient** though, because each time the sequence changes, one must **fully recompute all the activations** in the network (forward pass) to get the next word.

[4] For instance, by taking the average of the vector representations of the words contained in the sequence. It's also possible to concatenate the word-vectors, though this may significantly increase the size of the input. Word-vectors can be one-hot, or obtained from a pretrained word embedding mode like Word2Vec.

Feedforward Neural Nets as Language Models

- Pretty much like *n*-grams, feedforward NNs can be **fed with a vector representation of a sequence of words**,[4] and **return a probability distribution over the lexicon**.
- To get a probability distribution in the output, the last layer must have **as many units as the number of words in the lexicon**, and must come with a **softmax** activation.
- The softmax ensures that all the components of the last layer are mapped to $[0; 1]$, s.t. the low values are close to 0, the high values close to 1, and s.t. the components together sum to 1!
- Feedforward NNs are **a bit inefficient** though, because each time the sequence changes, one must **fully recompute all the activations** in the network (forward pass) to get the next word.

[4] For instance, by taking the average of the vector representations of the words contained in the sequence. It's also possible to concatenate the word-vectors, though this may significantly increase the size of the input. Word-vectors can be one-hot, or obtained from a pretrained word embedding mode like Word2Vec.

Feedforward Neural Nets as Language Models

- Pretty much like *n*-grams, feedforward NNs can be **fed with a vector representation of a sequence of words**,[4] and **return a probability distribution over the lexicon**.
- To get a probability distribution in the output, the last layer must have **as many units as the number of words in the lexicon**, and must come with a **softmax** activation.
- The softmax ensures that all the components of the last layer are mapped to $[0; 1]$, s.t. the low values are close to 0, the high values close to 1, and s.t. the components together sum to 1!
- Feedforward NNs are **a bit inefficient** though, because each time the sequence changes, one must **fully recompute all the activations** in the network (forward pass) to get the next word.

[4] For instance, by taking the average of the vector representations of the words contained in the sequence. It's also possible to concatenate the word-vectors, though this may significantly increase the size of the input. Word-vectors can be one-hot, or obtained from a pretrained word embedding mode like Word2Vec.

Feedforward Neural Nets as Language Models

- Pretty much like *n*-grams, feedforward NNs can be **fed with a vector representation of a sequence of words**,[4] and **return a probability distribution over the lexicon**.
- To get a probability distribution in the output, the last layer must have **as many units as the number of words in the lexicon**, and must come with a **softmax** activation.
- The softmax ensures that all the components of the last layer are mapped to $[0; 1]$, s.t. the low values are close to 0, the high values close to 1, and s.t. the components together sum to 1!
- Feedforward NNs are **a bit inefficient** though, because each time the sequence changes, one must **fully recompute all the activations** in the network (forward pass) to get the next word.

[4] For instance, by taking the average of the vector representations of the words contained in the sequence. It's also possible to concatenate the word-vectors, though this may significantly increase the size of the input. Word-vectors can be one-hot, or obtained from a pretrained word embedding mode like Word2Vec.

Feedforward example with averaged input

- Suppose our lexicon is of size 6, and we predict the next word from a sequence of 2 words. Our output will also have size 6, since it's the size of our lexicon. We encode our words in a one-hot fashion.

Feedforward example with averaged input

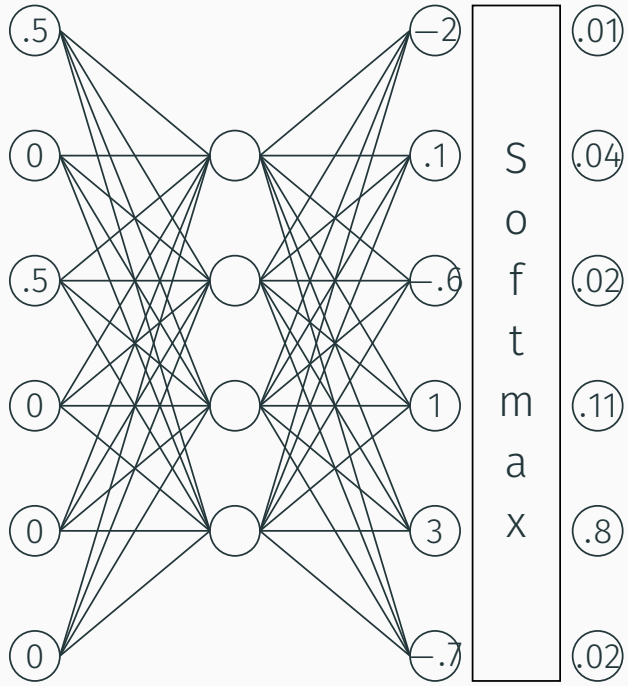
- Suppose our lexicon is of size 6, and we predict the next word from a sequence of 2 words. Our output will also have size 6, since it's the size of our lexicon. We encode our words in a one-hot fashion.
- Suppose our sequence is $[\textit{the}, \textit{cat}]$. *the* is encoded as $[1, 0, 0, 0, 0, 0]$ and *cat* as $[0, 0, 1, 0, 0, 0]$. We assume the entire sequence is represented by averaging the word-vectors contained in the sequence. The input is then

Feedforward example with averaged input

- Suppose our lexicon is of size 6, and we predict the next word from a sequence of 2 words. Our output will also have size 6, since it's the size of our lexicon. We encode our words in a one-hot fashion.
- Suppose our sequence is $[\textit{the}, \textit{cat}]$. *the* is encoded as $[1, 0, 0, 0, 0, 0]$ and *cat* as $[0, 0, 1, 0, 0, 0]$. We assume the entire sequence is represented by averaging the word-vectors contained in the sequence. The input is then $[.5, 0, .5, 0, 0, 0]$.

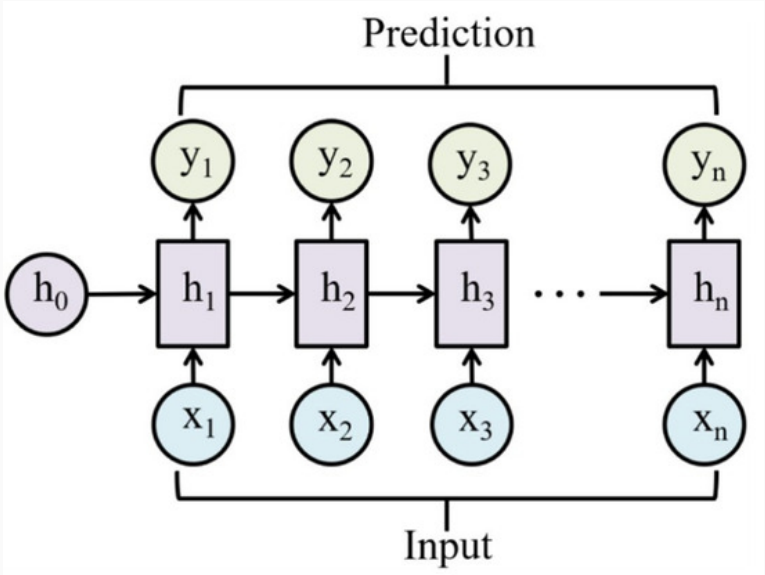
Feedforward example with averaged input

- Suppose our lexicon is of size 6, and we predict the next word from a sequence of 2 words. Our output will also have size 6, since it's the size of our lexicon. We encode our words in a one-hot fashion.
- Suppose our sequence is [*the*, *cat*]. *the* is encoded as [1, 0, 0, 0, 0, 0] and *cat* as [0, 0, 1, 0, 0, 0]. We assume the entire sequence is represented by averaging the word-vectors contained in the sequence. The input is then [.5, 0, .5, 0, 0, 0].



- After the forward pass and the softmax, we end up with a distribution favoring component 5, with probability .8.

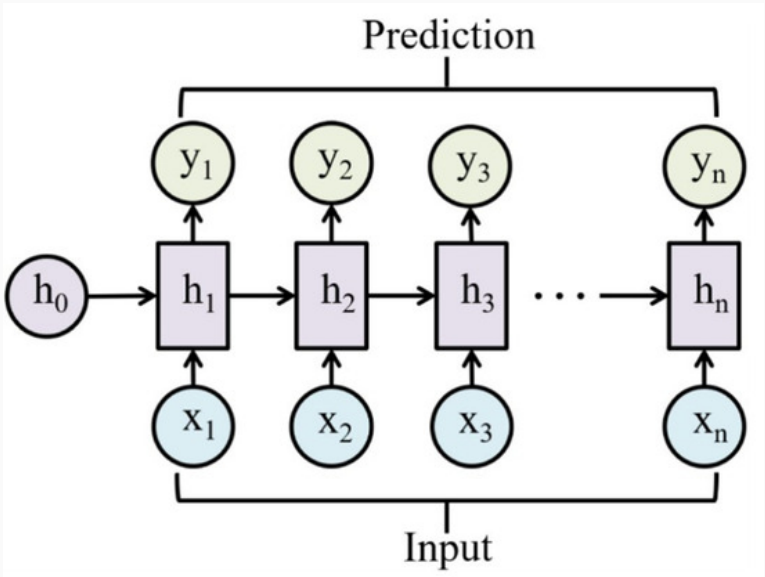
Recurrent Neural Nets as Language Models



- A RNN can be fed with **word vectors** (x_i) **one-by-one**, and once the initial "seed" sequence is all processed, the model can start producing a probability distribution over possible continuations.

- The continuations (the y_i above) are sampled and then fed back to the model.
- Again, just make sure the output vector has **matches the size of the lexicon**, and apply a **softmax** activation on top of it to get a probability distribution.
- RNNs are **more efficient than feedforward NNs** as Language Models: they can be easily "expanded" without recomputing the entire information flow, while preserving the hidden state which accumulates information from the start!

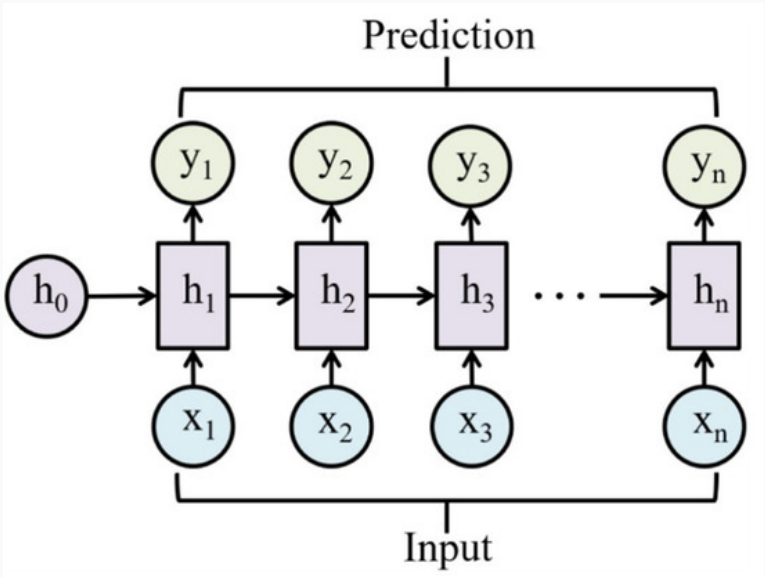
Recurrent Neural Nets as Language Models



- A RNN can be fed with **word vectors** (x_i) **one-by-one**, and once the initial "seed" sequence is all processed, the model can start producing a probability distribution over possible continuations.

- The continuations (the y_i above) are sampled and then fed back to the model.
- Again, just make sure the output vector has **matches the size of the lexicon**, and apply a **softmax** activation on top of it to get a probability distribution.
- RNNs are **more efficient than feedforward NNs** as Language Models: they can be easily "expanded" without recomputing the entire information flow, while preserving the hidden state which accumulates information from the start!

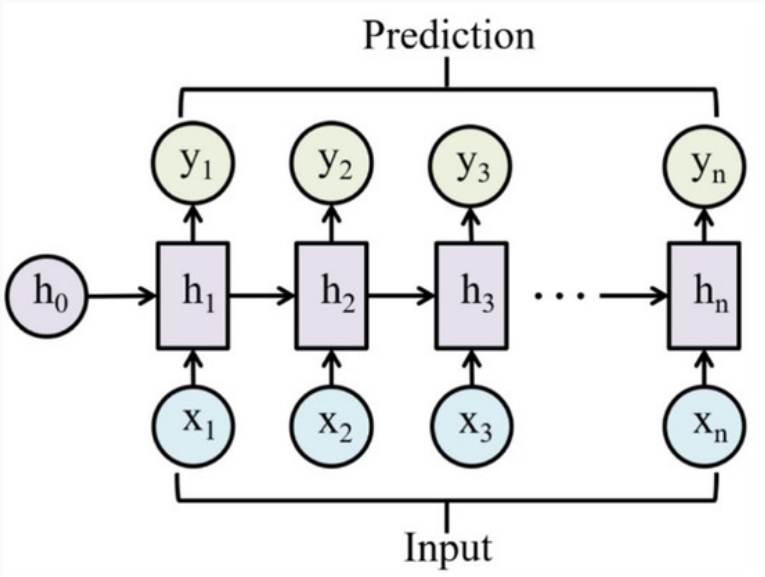
Recurrent Neural Nets as Language Models



- A RNN can be fed with **word vectors** (x_i) **one-by-one**, and once the initial "seed" sequence is all processed, the model can start producing a probability distribution over possible continuations.

- The continuations (the y_i above) are sampled and then fed back to the model.
- Again, just make sure the output vector has **matches the size of the lexicon**, and apply a **softmax** activation on top of it to get a probability distribution.
- RNNs are **more efficient than feedforward NNs** as Language Models: they can be easily "expanded" without recomputing the entire information flow, while preserving the hidden state which accumulates information from the start!

Recurrent Neural Nets as Language Models



- A RNN can be fed with **word vectors** (x_i) **one-by-one**, and once the initial "seed" sequence is all processed, the model can start producing a probability distribution over possible continuations.

- The continuations (the y_i above) are sampled and then fed back to the model.
- Again, just make sure the output vector has **matches the size of the lexicon**, and apply a **softmax** activation on top of it to get a probability distribution.
- RNNs are **more efficient than feedforward NNs** as Language Models: they can be easily "expanded" without recomputing the entire information flow, while preserving the hidden state which accumulates information from the start!

Large Language Models

Why “Large” Language Models

- Large Language Models are simply “big” Language Models: they **predict the next word/token** just like *n*-gram models, only they have **a lot of layers, parameters, activations**.
- Such models are deeply rooted in the **Distributional Hypothesis**, according to which the meaning of a word *w* is hugely dependent on which words cooccur with *w*.
- They derive **contextual and multi-level representations** about the sentence being processed, allowing them to generate continuations that are **meaningful for a wide range of tasks**.
- LLMs are characterized by:
  - A **tokenization procedure**, transforming the input sequence into a series of tokens (integers) interpretable by the model.
  - A certain type of **information flow**, i.e. ways in which the tokens in the sequence end up influencing each other’s representations.
- Many LLMs are based on the **Transformer** architecture; but we’ll remain architecture-neutral today.

Why “Large” Language Models

- Large Language Models are simply “big” Language Models: they **predict the next word/token** just like *n*-gram models, only they have **a lot of layers, parameters, activations**.
- Such models are deeply rooted in the **Distributional Hypothesis**, according to which the meaning of a word *w* is hugely dependent on which words cooccur with *w*.
- They derive **contextual and multi-level representations** about the sentence being processed, allowing them to generate continuations that are **meaningful for a wide range of tasks**.
- LLMs are characterized by:
  - A **tokenization procedure**, transforming the input sequence into a series of tokens (integers) interpretable by the model.
  - A certain type of **information flow**, i.e. ways in which the tokens in the sequence end up influencing each other’s representations.
- Many LLMs are based on the **Transformer** architecture; but we’ll remain architecture-neutral today.

Why “Large” Language Models

- Large Language Models are simply “big” Language Models: they **predict the next word/token** just like *n*-gram models, only they have **a lot of layers, parameters, activations**.
- Such models are deeply rooted in the **Distributional Hypothesis**, according to which the meaning of a word *w* is hugely dependent on which words cooccur with *w*.
- They derive **contextual and multi-level representations** about the sentence being processed, allowing them to generate continuations that are **meaningful for a wide range of tasks**.
- LLMs are characterized by:
  - A **tokenization procedure**, transforming the input sequence into a series of tokens (integers) interpretable by the model.
  - A certain type of **information flow**, i.e. ways in which the tokens in the sequence end up influencing each other’s representations.
- Many LLMs are based on the **Transformer** architecture; but we’ll remain architecture-neutral today.

Why “Large” Language Models

- Large Language Models are simply “big” Language Models: they **predict the next word/token** just like *n*-gram models, only they have **a lot of layers, parameters, activations**.
- Such models are deeply rooted in the **Distributional Hypothesis**, according to which the meaning of a word *w* is hugely dependent on which words cooccur with *w*.
- They derive **contextual and multi-level representations** about the sentence being processed, allowing them to generate continuations that are **meaningful for a wide range of tasks**.
- LLMs are characterized by:
  - A **tokenization procedure**, transforming the input sequence into a series of tokens (integers) interpretable by the model.
  - A certain type of **information flow**, i.e. ways in which the tokens in the sequence end up influencing each other’s representations.
- Many LLMs are based on the **Transformer** architecture; but we’ll remain architecture-neutral today.

Why “Large” Language Models

- Large Language Models are simply “big” Language Models: they **predict the next word/token** just like *n*-gram models, only they have **a lot of layers, parameters, activations**.
- Such models are deeply rooted in the **Distributional Hypothesis**, according to which the meaning of a word *w* is hugely dependent on which words cooccur with *w*.
- They derive **contextual and multi-level representations** about the sentence being processed, allowing them to generate continuations that are **meaningful for a wide range of tasks**.
- LLMs are characterized by:
  - A **tokenization procedure**, transforming the input sequence into a series of tokens (integers) interpretable by the model.
  - A certain type of **information flow**, i.e. ways in which the tokens in the sequence end up influencing each other’s representations.
- Many LLMs are based on the **Transformer** architecture; but we’ll remain architecture-neutral today.

Why “Large” Language Models

- Large Language Models are simply “big” Language Models: they **predict the next word/token** just like *n*-gram models, only they have **a lot of layers, parameters, activations**.
- Such models are deeply rooted in the **Distributional Hypothesis**, according to which the meaning of a word *w* is hugely dependent on which words cooccur with *w*.
- They derive **contextual and multi-level representations** about the sentence being processed, allowing them to generate continuations that are **meaningful for a wide range of tasks**.
- LLMs are characterized by:
  - A **tokenization procedure**, transforming the input sequence into a series of tokens (integers) interpretable by the model.
  - A certain type of **information flow**, i.e. ways in which the tokens in the sequence end up influencing each other’s representations.
- Many LLMs are based on the **Transformer** architecture; but we’ll remain architecture-neutral today.

Why “Large” Language Models

- Large Language Models are simply “big” Language Models: they **predict the next word/token** just like *n*-gram models, only they have **a lot of layers, parameters, activations**.
- Such models are deeply rooted in the **Distributional Hypothesis**, according to which the meaning of a word *w* is hugely dependent on which words cooccur with *w*.
- They derive **contextual and multi-level representations** about the sentence being processed, allowing them to generate continuations that are **meaningful for a wide range of tasks**.
- LLMs are characterized by:
  - A **tokenization procedure**, transforming the input sequence into a series of tokens (integers) interpretable by the model.
  - A certain type of **information flow**, i.e. ways in which the tokens in the sequence end up influencing each other’s representations.
- Many LLMs are based on the **Transformer** architecture; but we’ll remain architecture-neutral today.

Why “Large” Language Models

- Large Language Models are simply “big” Language Models: they **predict the next word/token** just like *n*-gram models, only they have **a lot of layers, parameters, activations**.
- Such models are deeply rooted in the **Distributional Hypothesis**, according to which the meaning of a word *w* is hugely dependent on which words cooccur with *w*.
- They derive **contextual and multi-level representations** about the sentence being processed, allowing them to generate continuations that are **meaningful for a wide range of tasks**.
- LLMs are characterized by:
  - A **tokenization procedure**, transforming the input sequence into a series of tokens (integers) interpretable by the model.
  - A certain type of **information flow**, i.e. ways in which the tokens in the sequence end up influencing each other’s representations.
- Many LLMs are based on the **Transformer** architecture; but we’ll remain architecture-neutral today.

Tokenization

- LLMs do not exactly generate the next word of a sequence. They reason on **tokens**–typically integer encodings of words or subwords.
- So, from a sequence of tokens, a model like GPT will produce a **probability distribution over the next token**. The actual next token will be generated by sampling.
- You can tokenize words online, to see the kind of input LLMs "see".
- For instance, token-ization is represented by GPT-4 as [5963, 2065, 1432]; distribution-al as [82816, 278, 1432]; h-ypo-thesis as [71, 60400, 13491, 1432]. The last token, 1432, is the **end-of-sequence** token.

Tokenization

- LLMs do not exactly generate the next word of a sequence. They reason on **tokens**–typically integer encodings of words or subwords.
- So, from a sequence of tokens, a model like GPT will produce a **probability distribution over the next token**. The actual next token will be generated by sampling.
- You can tokenize words online, to see the kind of input LLMs "see".
- For instance, token-ization is represented by GPT-4 as [5963, 2065, 1432]; distribution-al as [82816, 278, 1432]; h-ypo-thesis as [71, 60400, 13491, 1432]. The last token, 1432, is the **end-of-sequence** token.

Tokenization

- LLMs do not exactly generate the next word of a sequence. They reason on **tokens**–typically integer encodings of words or subwords.
- So, from a sequence of tokens, a model like GPT will produce a **probability distribution over the next token**. The actual next token will be generated by sampling.
- You can tokenize words online, to see the kind of input LLMs "see".
- For instance, token-ization is represented by GPT-4 as [5963, 2065, 1432]; distribution-al as [82816, 278, 1432]; h-ypo-thesis as [71, 60400, 13491, 1432]. The last token, 1432, is the **end-of-sequence** token.

Tokenization

- LLMs do not exactly generate the next word of a sequence. They reason on **tokens**–typically integer encodings of words or subwords.
- So, from a sequence of tokens, a model like GPT will produce a **probability distribution over the next token**. The actual next token will be generated by sampling.
- You can tokenize words online, to see the kind of input LLMs "see".
- For instance, token-ization is represented by GPT-4 as [5963, 2065, 1432]; distribution-al as [82816, 278, 1432]; h-ypo-thesis as [71, 60400, 13491, 1432]. The last token, 1432, is the **end-of-sequence** token.

Tokenization procedure: Byte-Pair Encoding

- Tokenizers are built based on **statistical regularities** observed in the words/sentences of the language.
- Byte-Pair Encoding (**BPE**) for instance, is a tokenization procedure which iteratively "glues" together sequences of characters that cooccur often. Common words Proper linguistic morphemes will tend to be represented as tokens.
- Here is how BPE works:
  - Start with the **most fine-grained tokenizer**, which treats each individual character (space included) as a separate token.
  - Scan the training corpus and find the two tokens with highest coocurrence; **"glue" them to form a new token**.
  - Stop after a certain number of "gluing" steps, or once a target number of tokens is reached, or when none of the coocurrences computed is more than 1.
- Any new text can be tokenized by repeating the "gluing" rules learned from the training data.

Tokenization procedure: Byte-Pair Encoding

- Tokenizers are built based on **statistical regularities** observed in the words/sentences of the language.
- Byte-Pair Encoding (**BPE**) for instance, is a tokenization procedure which iteratively "glues" together sequences of characters that cooccur often. Common words Proper linguistic morphemes will tend to be represented as tokens.
- Here is how BPE works:
  - Start with the **most fine-grained tokenizer**, which treats each individual character (space included) as a separate token.
  - Scan the training corpus and find the two tokens with highest coocurrence; **"glue" them to form a new token**.
  - Stop after a certain number of "gluing" steps, or once a target number of tokens is reached, or when none of the coocurrences computed is more than 1.
- Any new text can be tokenized by repeating the "gluing" rules learned from the training data.

Tokenization procedure: Byte-Pair Encoding

- Tokenizers are built based on **statistical regularities** observed in the words/sentences of the language.
- Byte-Pair Encoding (**BPE**) for instance, is a tokenization procedure which iteratively "glues" together sequences of characters that cooccur often. Common words Proper linguistic morphemes will tend to be represented as tokens.
- Here is how BPE works:
  - Start with the **most fine-grained tokenizer**, which treats each individual character (space included) as a separate token.
  - Scan the training corpus and find the two tokens with highest coocurrence; **"glue" them to form a new token**.
  - Stop after a certain number of "gluing" steps, or once a target number of tokens is reached, or when none of the coocurrences computed is more than 1.
- Any new text can be tokenized by repeating the "gluing" rules learned from the training data.

Tokenization procedure: Byte-Pair Encoding

- Tokenizers are built based on **statistical regularities** observed in the words/sentences of the language.
- Byte-Pair Encoding (**BPE**) for instance, is a tokenization procedure which iteratively "glues" together sequences of characters that cooccur often. Common words Proper linguistic morphemes will tend to be represented as tokens.
- Here is how BPE works:
  - Start with the **most fine-grained tokenizer**, which treats each individual character (space included) as a separate token.
  - Scan the training corpus and find the two tokens with highest coocurrence; **"glue" them to form a new token**.
  - Stop after a certain number of "gluing" steps, or once a target number of tokens is reached, or when none of the coocurrences computed is more than 1.
- Any new text can be tokenized by repeating the "gluing" rules learned from the training data.

Tokenization procedure: Byte-Pair Encoding

- Tokenizers are built based on **statistical regularities** observed in the words/sentences of the language.
- Byte-Pair Encoding (**BPE**) for instance, is a tokenization procedure which iteratively "glues" together sequences of characters that cooccur often. Common words Proper linguistic morphemes will tend to be represented as tokens.
- Here is how BPE works:
  - Start with the **most fine-grained tokenizer**, which treats each individual character (space included) as a separate token.
  - Scan the training corpus and find the two tokens with highest coocurrence; **"glue" them to form a new token**.
  - Stop after a certain number of "gluing" steps, or once a target number of tokens is reached, or when none of the coocurrences computed is more than 1.
- Any new text can be tokenized by repeating the "gluing" rules learned from the training data.

Tokenization procedure: Byte-Pair Encoding

- Tokenizers are built based on **statistical regularities** observed in the words/sentences of the language.
- Byte-Pair Encoding (**BPE**) for instance, is a tokenization procedure which iteratively "glues" together sequences of characters that cooccur often. Common words Proper linguistic morphemes will tend to be represented as tokens.
- Here is how BPE works:
  - Start with the **most fine-grained tokenizer**, which treats each individual character (space included) as a separate token.
  - Scan the training corpus and find the two tokens with highest coocurrence; **"glue" them to form a new token**.
  - Stop after a certain number of "gluing" steps, or once a target number of tokens is reached, or when none of the coocurrences computed is more than 1.
- Any new text can be tokenized by repeating the "gluing" rules learned from the training data.

Tokenization procedure: Byte-Pair Encoding

- Tokenizers are built based on **statistical regularities** observed in the words/sentences of the language.
- Byte-Pair Encoding (**BPE**) for instance, is a tokenization procedure which iteratively "glues" together sequences of characters that cooccur often. Common words Proper linguistic morphemes will tend to be represented as tokens.
- Here is how BPE works:
  - Start with the **most fine-grained tokenizer**, which treats each individual character (space included) as a separate token.
  - Scan the training corpus and find the two tokens with highest coocurrence; **"glue" them to form a new token**.
  - Stop after a certain number of "gluing" steps, or once a target number of tokens is reached, or when none of the coocurrences computed is more than 1.
- Any new text can be tokenized by repeating the "gluing" rules learned from the training data.

Tokenization practice

- BPE glues together characters that coocur frequently. What kind of tokens do you think will be produced from the following dataset? Which triplets of adjectives will be properly tokenized?

(1) low lower lowest strong stronger strongest good better best

- Let's try to see how this online BPE visualizer deals with this adjectives.

Tokenization practice

- BPE glues together characters that coocur frequently. What kind of tokens do you think will be produced from the following dataset? Which triplets of adjectives will be properly tokenized?

(2) low lower lowest strong stronger strongest good better best

- Let's try to see how this online BPE visualizer deals with this adjectives.

Core architectures of LLMs

- LLMs can implement different kinds of information flow: **encoder**-only, **decoder**-only, or **encoder-decoder** models.
- Each type of architecture is made for a specific family of tasks, and produces different kinds of outputs.
- The **Decoder architecture is primarily used for Language Modeling**, i.e. next token prediction. The two other architectures can also be used for this, but not as readily.

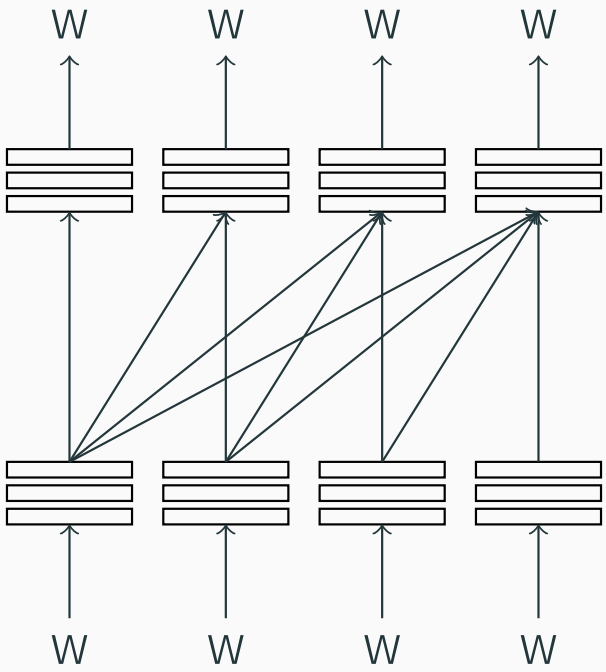
Core architectures of LLMs

- LLMs can implement different kinds of information flow: **encoder**-only, **decoder**-only, or **encoder-decoder** models.
- Each type of architecture is made for a specific family of tasks, and produces different kinds of outputs.
- The **Decoder architecture is primarily used for Language Modeling**, i.e. next token prediction. The two other architectures can also be used for this, but not as readily.

Core architectures of LLMs

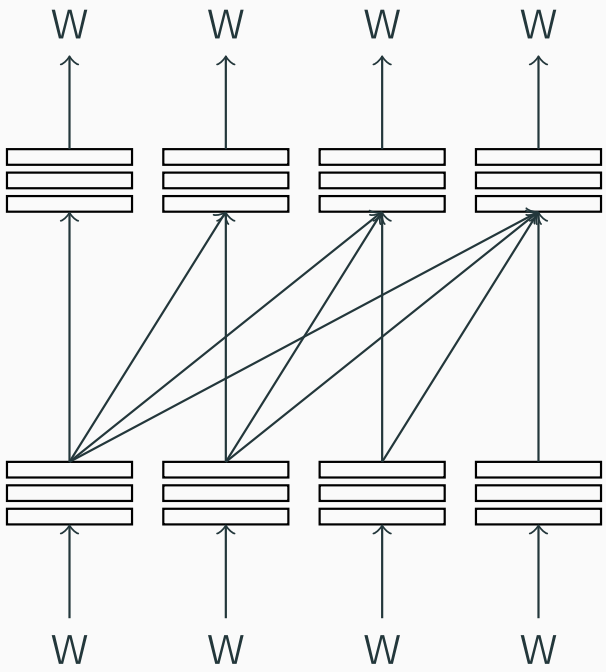
- LLMs can implement different kinds of information flow: **encoder**-only, **decoder**-only, or **encoder-decoder** models.
- Each type of architecture is made for a specific family of tasks, and produces different kinds of outputs.
- The **Decoder architecture is primarily used for Language Modeling**, i.e. next token prediction. The two other architectures can also be used for this, but not as readily.

The Decoder model



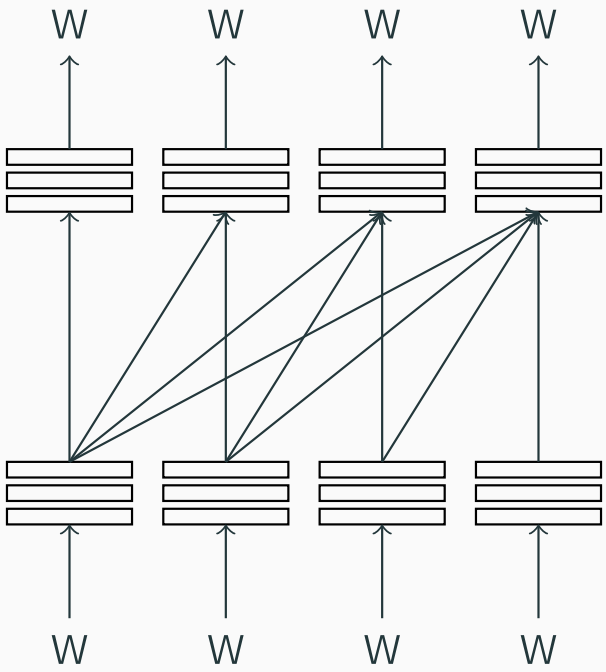
- Takes as input a **series of tokens**, and **iteratively generates an output token** one at a time.
- The stacked rectangles represent a **series of linear and non-linear operations** (think small neural nets, RNN or LSTM modules).
- Information flows strictly **left-to-right** (also called causal/autoregressive).
- GPT, Claude, Llama, and Mistral use the Decoder model.
- Training based on **next-word prediction**.

The Decoder model



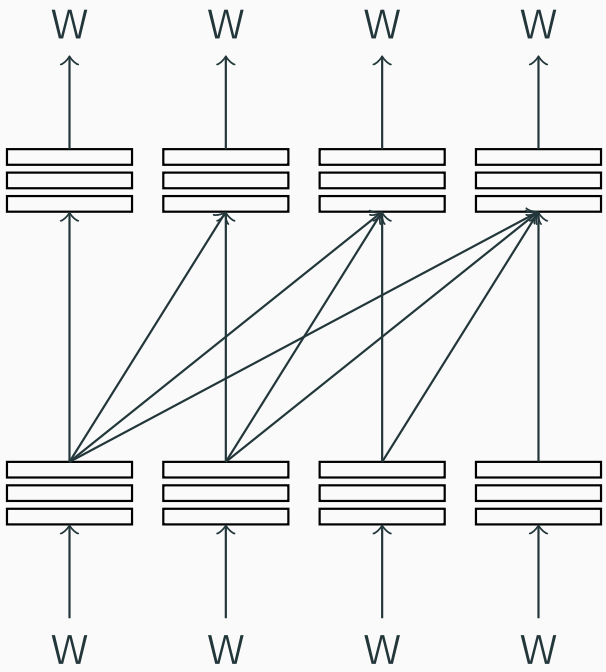
- Takes as input a **series of tokens**, and **iteratively generates an output token** one at a time.
- The stacked rectangles represent a **series of linear and non-linear operations** (think small neural nets, RNN or LSTM modules).
- Information flows strictly **left-to-right** (also called causal/autoregressive).
- GPT, Claude, Llama, and Mistral use the Decoder model.
- Training based on **next-word prediction**.

The Decoder model



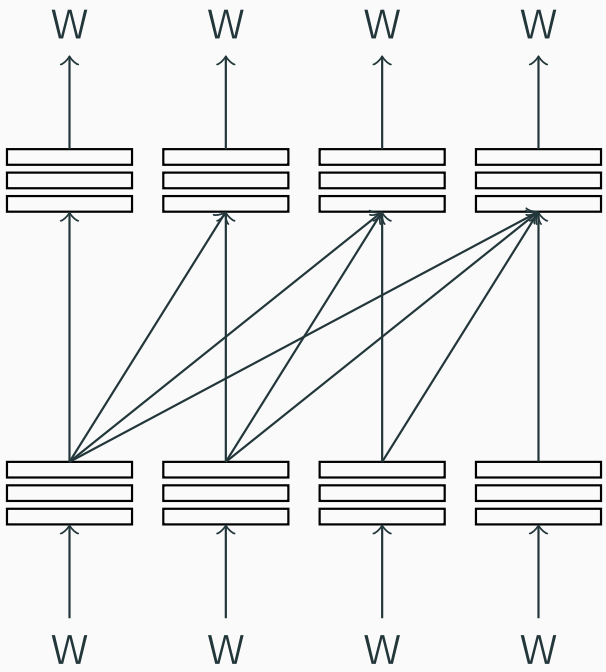
- Takes as input a **series of tokens**, and **iteratively generates an output token** one at a time.
- The stacked rectangles represent a **series of linear and non-linear operations** (think small neural nets, RNN or LSTM modules).
- Information flows strictly **left-to-right** (also called causal/autoregressive).
- GPT, Claude, Llama, and Mistral use the Decoder model.
- Training based on **next-word prediction**.

The Decoder model



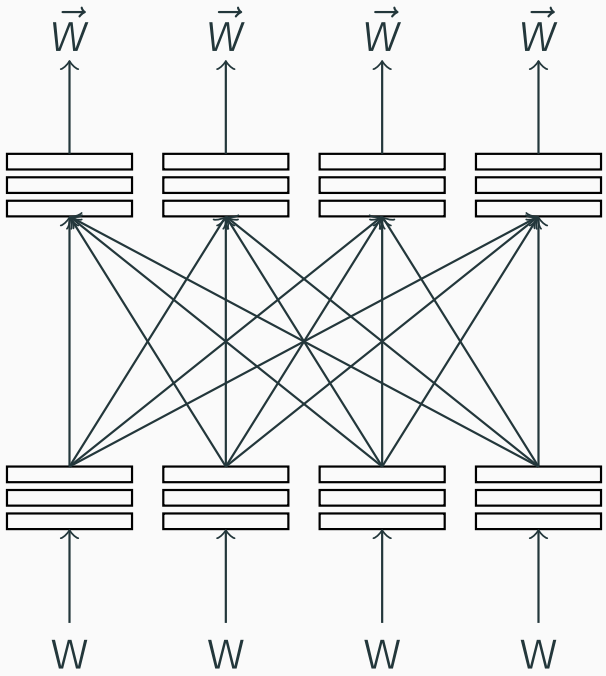
- Takes as input a **series of tokens**, and **iteratively generates an output token** one at a time.
- The stacked rectangles represent a **series of linear and non-linear operations** (think small neural nets, RNN or LSTM modules).
- Information flows strictly **left-to-right** (also called causal/autoregressive).
- GPT, Claude, Llama, and Mistral use the Decoder model.
- Training based on **next-word prediction**.

The Decoder model



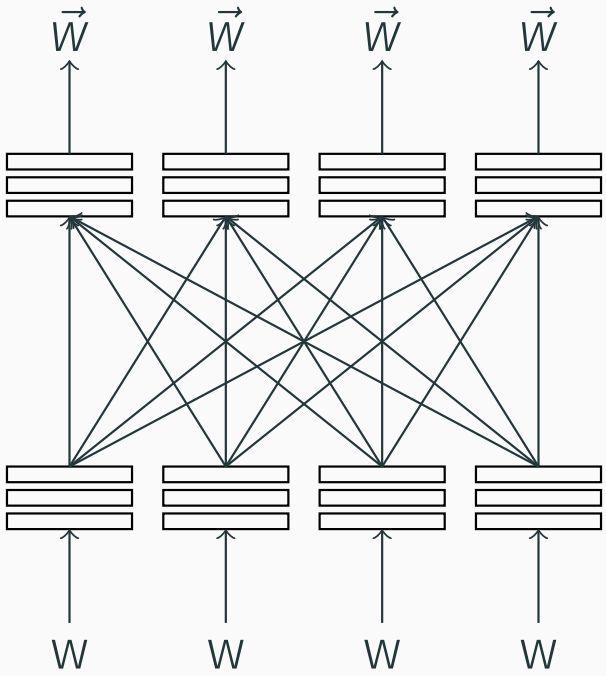
- Takes as input a **series of tokens**, and **iteratively generates an output token** one at a time.
- The stacked rectangles represent a **series of linear and non-linear operations** (think small neural nets, RNN or LSTM modules).
- Information flows strictly **left-to-right** (also called causal/autoregressive).
- GPT, Claude, Llama, and Mistral use the Decoder model.
- Training based on **next-word prediction**.

The Encoder model



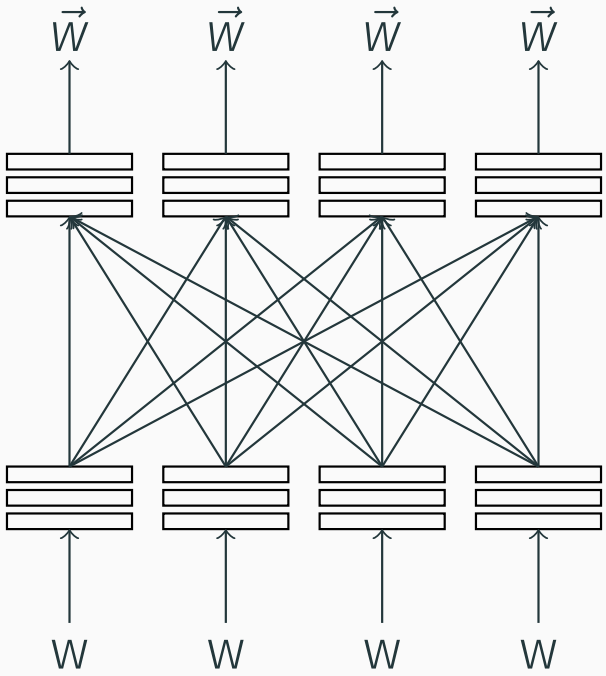
- Takes as input a **series of tokens**, and generates **contextual word-vectors**, that can be used for downstream tasks (e.g. classification). Not a pure Language Model!
- Information flows **bidirectionally**.
- Behind BERT and all its variants.
- Training based on **Masked Language Modeling** (predict a masked word anywhere in the sentence, based on its left and right context).

The Encoder model



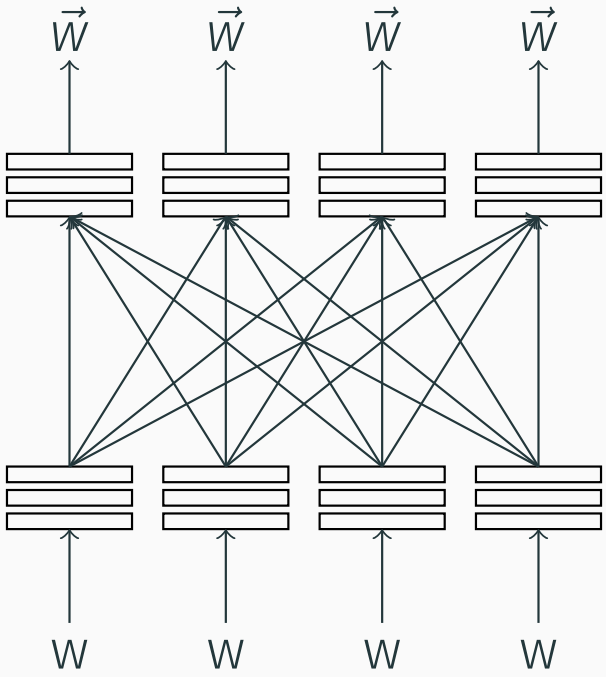
- Takes as input a **series of tokens**, and generates **contextual word-vectors**, that can be used for downstream tasks (e.g. classification). Not a pure Language Model!
- Information flows **bidirectionally**.
- Behind BERT and all its variants.
- Training based on **Masked Language Modeling** (predict a masked word anywhere in the sentence, based on its left and right context).

The Encoder model



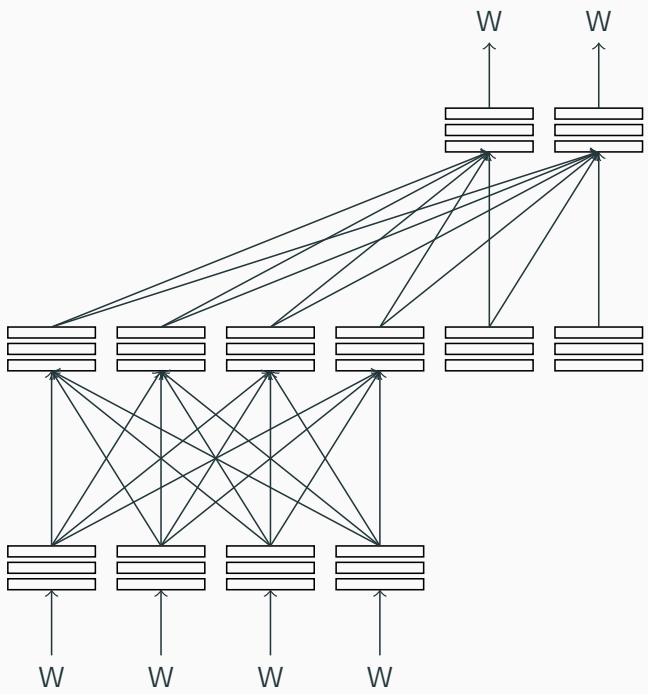
- Takes as input a **series of tokens**, and generates **contextual word-vectors**, that can be used for downstream tasks (e.g. classification). Not a pure Language Model!
- Information flows **bidirectionally**.
- Behind BERT and all its variants.
- Training based on **Masked Language Modeling** (predict a masked word anywhere in the sentence, based on its left and right context).

The Encoder model



- Takes as input a **series of tokens**, and generates **contextual word-vectors**, that can be used for downstream tasks (e.g. classification). Not a pure Language Model!
- Information flows **bidirectionally**.
- Behind BERT and all its variants.
- Training based on **Masked Language Modeling** (predict a masked word anywhere in the sentence, based on its left and right context).

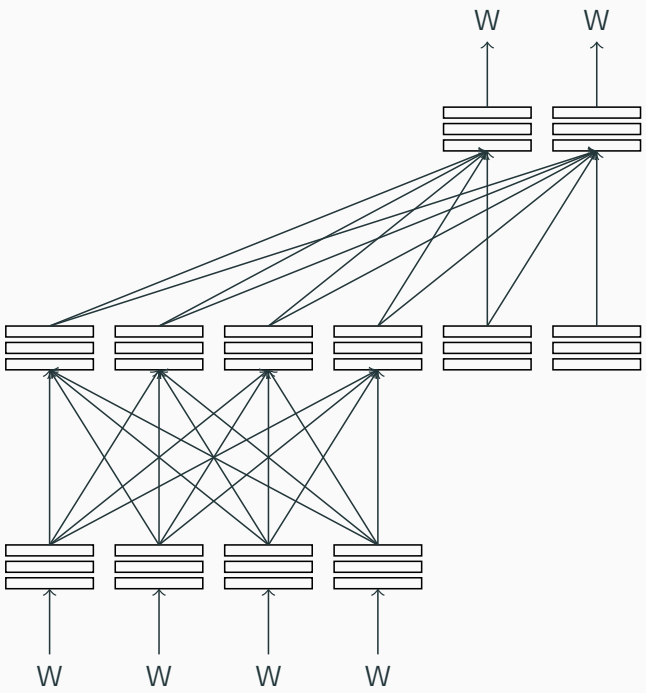
The Encoder-Decoder model



- Takes as input a **series of tokens**, and outputs another **series of tokens**–similar to the Seq2Seq model seen last week.
- First, information flows bidirectionally, to generate an "hidden" contextual vector representation of the sequence.
- Then, information flows left-to-right to generate the output sequence from the hidden state (and previous output fed back to the model).

- Behind BART, T5.
- Used for tasks involving a **"loose" input-output relationship**, e.g. machine translation, speech recognition...

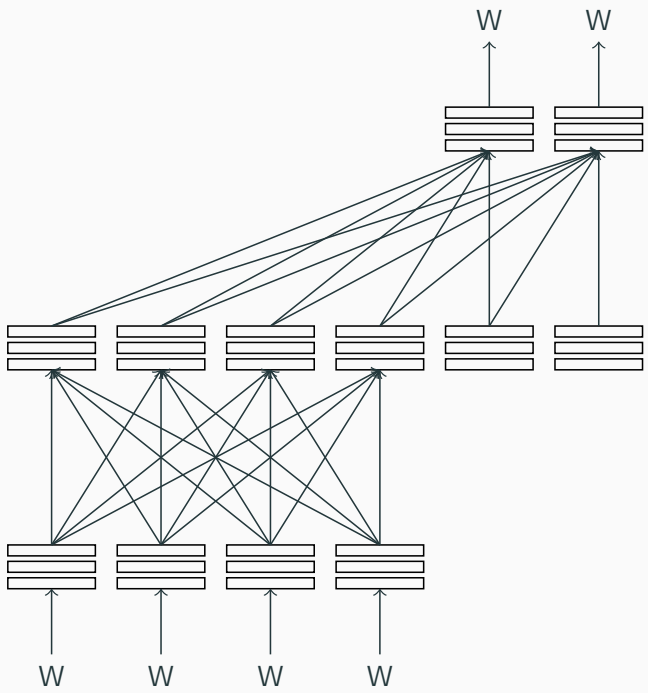
The Encoder-Decoder model



- Takes as input a **series of tokens**, and outputs another **series of tokens**–similar to the Seq2Seq model seen last week.
- First, information flows bidirectionally, to generate an "hidden" contextual vector representation of the sequence.
- Then, information flows left-to-right to generate the output sequence from the hidden state (and previous output fed back to the model).

- Behind BART, T5.
- Used for tasks involving a **"loose" input-output relationship**, e.g. machine translation, speech recognition...

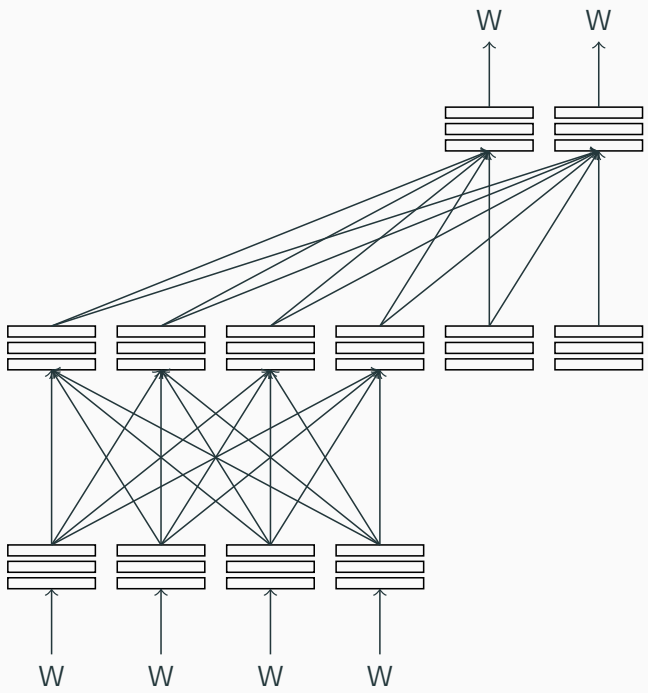
The Encoder-Decoder model



- Takes as input a **series of tokens**, and outputs another **series of tokens**–similar to the Seq2Seq model seen last week.
- First, information flows bidirectionally, to generate an "hidden" contextual vector representation of the sequence.
- Then, information flows left-to-right to generate the output sequence from the hidden state (and previous output fed back to the model).

- Behind BART, T5.
- Used for tasks involving a **"loose" input-output relationship**, e.g. machine translation, speech recognition...

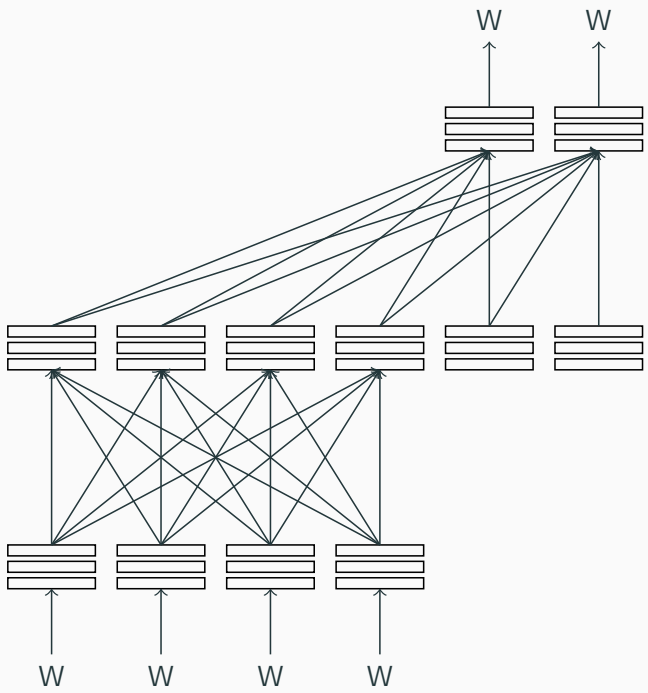
The Encoder-Decoder model



- Takes as input a **series of tokens**, and outputs another **series of tokens**–similar to the Seq2Seq model seen last week.
- First, information flows bidirectionally, to generate an "hidden" contextual vector representation of the sequence.
- Then, information flows left-to-right to generate the output sequence from the hidden state (and previous output fed back to the model).

- Behind BART, T5.
- Used for tasks involving a **"loose" input-output relationship**, e.g. machine translation, speech recognition...

The Encoder-Decoder model



- Takes as input a **series of tokens**, and outputs another **series of tokens**–similar to the Seq2Seq model seen last week.
- First, information flows bidirectionally, to generate an "hidden" contextual vector representation of the sequence.
- Then, information flows left-to-right to generate the output sequence from the hidden state (and previous output fed back to the model).

- Behind BART, T5.
- Used for tasks involving a **"loose" input-output relationship**, e.g. machine translation, speech recognition...

Training LLMs

Three main training steps

- LLMs are trained in three main phases: general pretraining, instruction-tuning, alignment.
- In the general **pretraining** phase, LLM Decoders are trained to predict the next token on general data from the Internet. In doing so, they **internalize a lot of useful linguistic patterns**: how sentences are built, what words mean in context...
- In the **instruction-tuning** phase, they learn to perform specific formatted tasks, e.g. question-answering, based on a few exemplars.
- In the **alignment** phase, they are trained to be maximally helpful and less harmful.

Three main training steps

- LLMs are trained in three main phases: general pretraining, instruction-tuning, alignment.
- In the general **pretraining** phase, LLM Decoders are trained to predict the next token on general data from the Internet. In doing so, they **internalize a lot of useful linguistic patterns**: how sentences are built, what words mean in context...
- In the **instruction-tuning** phase, they learn to perform specific formatted tasks, e.g. question-answering, based on a few exemplars.
- In the **alignment** phase, they are trained to be maximally helpful and less harmful.

Three main training steps

- LLMs are trained in three main phases: general pretraining, instruction-tuning, alignment.
- In the general **pretraining** phase, LLM Decoders are trained to predict the next token on general data from the Internet. In doing so, they **internalize a lot of useful linguistic patterns**: how sentences are built, what words mean in context...
- In the **instruction-tuning** phase, they learn to perform specific formatted tasks, e.g. question-answering, based on a few exemplars.
- In the **alignment** phase, they are trained to be maximally helpful and less harmful.

General pretraining

- Pretraining is based on **general data from the Internet** (academic publications, public domain books, forums, Wikipedia, open-source codebases...). This produces a huge sequence of text that can be tokenized.
- Based on a large context from this massive dataset, the model is trained to predict the next token in a **self-supervised way** (no need to label data!)
- Whatever the model predicts at each training step is *not* used for further training; the training sequence remains faithful to the data. For instance, if the model predicts *flower* to follow *a beautiful*, and the right continuation was *painting*, the context *a beautiful painting* (and not *a beautiful flower*) will be used to predict the following token. This is known as **teacher forcing**.
- The data used for pretraining is sometimes **problematic**: it may violate copyright laws, user privacy, and it may contain biased or even toxic content.

General pretraining

- Pretraining is based on **general data from the Internet** (academic publications, public domain books, forums, Wikipedia, open-source codebases...). This produces a huge sequence of text that can be tokenized.
- Based on a large context from this massive dataset, the model is trained to predict the next token in a **self-supervised way** (no need to label data!)
- Whatever the model predicts at each training step is *not* used for further training; the training sequence remains faithful to the data. For instance, if the model predicts *flower* to follow *a beautiful*, and the right continuation was *painting*, the context *a beautiful painting* (and not *a beautiful flower*) will be used to predict the following token. This is known as **teacher forcing**.
- The data used for pretraining is sometimes **problematic**: it may violate copyright laws, user privacy, and it may contain biased or even toxic content.

General pretraining

- Pretraining is based on **general data from the Internet** (academic publications, public domain books, forums, Wikipedia, open-source codebases...). This produces a huge sequence of text that can be tokenized.
- Based on a large context from this massive dataset, the model is trained to predict the next token in a **self-supervised way** (no need to label data!)
- Whatever the model predicts at each training step is *not* used for further training; the training sequence remains faithful to the data. For instance, if the model predicts *flower* to follow *a beautiful*, and the right continuation was *painting*, the context *a beautiful painting* (and not *a beautiful flower*) will be used to predict the following token. This is known as **teacher forcing**.
- The data used for pretraining is sometimes **problematic**: it may violate copyright laws, user privacy, and it may contain biased or even toxic content.

General pretraining

- Pretraining is based on **general data from the Internet** (academic publications, public domain books, forums, Wikipedia, open-source codebases...). This produces a huge sequence of text that can be tokenized.
- Based on a large context from this massive dataset, the model is trained to predict the next token in a **self-supervised way** (no need to label data!)
- Whatever the model predicts at each training step is *not* used for further training; the training sequence remains faithful to the data. For instance, if the model predicts *flower* to follow *a beautiful*, and the right continuation was *painting*, the context *a beautiful painting* (and not *a beautiful flower*) will be used to predict the following token. This is known as **teacher forcing**.
- The data used for pretraining is sometimes **problematic**: it may violate copyright laws, user privacy, and it may contain biased or even toxic content.

Instruction-tuning

- Pretrained LLMs can be **fine-tuned on a more specific dataset to perform a specific task**, e.g. question answering, arithmetic, grammatical error correction... This is called **instruction-tuning**.
- The goal is to retain domain-general linguistic knowledge, while securing more task-specific continuations.
- To achieve this, the LLM gets trained further on a special dataset of instructions together with the appropriate response to each. For instance, question-answer pairs; math problems and how to solve them.
- Instruction-tuning **slightly updates the internal parameters of the model** (think: its many many βs).

Instruction-tuning

- Pretrained LLMs can be **fine-tuned on a more specific dataset to perform a specific task**, e.g. question answering, arithmetic, grammatical error correction... This is called **instruction-tuning**.
- The goal is to retain domain-general linguistic knowledge, while securing more task-specific continuations.
- To achieve this, the LLM gets trained further on a special dataset of instructions together with the appropriate response to each. For instance, question-answer pairs; math problems and how to solve them.
- Instruction-tuning **slightly updates the internal parameters of the model** (think: its many many βs).

Instruction-tuning

- Pretrained LLMs can be **fine-tuned on a more specific dataset to perform a specific task**, e.g. question answering, arithmetic, grammatical error correction... This is called **instruction-tuning**.
- The goal is to retain domain-general linguistic knowledge, while securing more task-specific continuations.
- To achieve this, the LLM gets trained further on a special dataset of instructions together with the appropriate response to each. For instance, question-answer pairs; math problems and how to solve them.
- Instruction-tuning **slightly updates the internal parameters of the model** (think: its many many βs).

Instruction-tuning

- Pretrained LLMs can be **fine-tuned on a more specific dataset to perform a specific task**, e.g. question answering, arithmetic, grammatical error correction... This is called **instruction-tuning**.
- The goal is to retain domain-general linguistic knowledge, while securing more task-specific continuations.
- To achieve this, the LLM gets trained further on a special dataset of instructions together with the appropriate response to each. For instance, question-answer pairs; math problems and how to solve them.
- Instruction-tuning **slightly updates the internal parameters of the model** (think: its many many βs).

Alignment

- During the alignment phase, the biases and harmful aspects of the training dataset are mitigated, and the quality of the responses is optimized according to user preferences.
- The model is given **preference data**, i.e. a context followed by **two potential continuations**, which are labeled (usually by people) as accepted or rejected.
- The model is then trained, by **reinforcement learning**[5] or other reward-based algorithms, to produce the accepted continuation and not the rejected continuation.

[5] Reinforcement Learning (RL) is a type of machine learning where an agent (e.g. a LLM) learns to make decisions by interacting with an environment, taking actions, and receiving feedback as rewards or penalties.

Alignment

- During the alignment phase, the biases and harmful aspects of the training dataset are mitigated, and the quality of the responses is optimized according to user preferences.
- The model is given **preference data**, i.e. a context followed by **two potential continuations**, which are labeled (usually by people) as accepted or rejected.
- The model is then trained, by **reinforcement learning**[5] or other reward-based algorithms, to produce the accepted continuation and not the rejected continuation.

[5] Reinforcement Learning (RL) is a type of machine learning where an agent (e.g. a LLM) learns to make decisions by interacting with an environment, taking actions, and receiving feedback as rewards or penalties.

Alignment

- During the alignment phase, the biases and harmful aspects of the training dataset are mitigated, and the quality of the responses is optimized according to user preferences.
- The model is given **preference data**, i.e. a context followed by **two potential continuations**, which are labeled (usually by people) as accepted or rejected.
- The model is then trained, by **reinforcement learning**[5] or other reward-based algorithms, to produce the accepted continuation and not the rejected continuation.

[5] Reinforcement Learning (RL) is a type of machine learning where an agent (e.g. a LLM) learns to make decisions by interacting with an environment, taking actions, and receiving feedback as rewards or penalties.

Tuning LLMs post-training

Prompt engineering

- Responses can also be optimized **after training**, by providing the model with contextual information about how the answer should look like. This is called **prompt engineering**.
- For instance, one can clearly specify the format of the desired answer in the prompt, give multiple choices, or even provide the model with one or more complete examples of what we are looking for (a demonstration). The latter approach is known as **few-shot learning**.
- The number of demonstrations doesn't need to be large; more examples seem to give diminishing returns, and can lead to overfitting. Demonstrations that have incorrect answers can still improve a system!
- Prompt engineering does not affect the weights of the model. In providing extra context, it **only affects the activation of the neurons**.

Prompt engineering

- Responses can also be optimized **after training**, by providing the model with contextual information about how the answer should look like. This is called **prompt engineering**.
- For instance, one can clearly specify the format of the desired answer in the prompt, give multiple choices, or even provide the model with one or more complete examples of what we are looking for (a demonstration). The latter approach is known as **few-shot learning**.
- The number of demonstrations doesn't need to be large; more examples seem to give diminishing returns, and can lead to overfitting. Demonstrations that have incorrect answers can still improve a system!
- Prompt engineering does not affect the weights of the model. In providing extra context, it **only affects the activation of the neurons**.

Prompt engineering

- Responses can also be optimized **after training**, by providing the model with contextual information about how the answer should look like. This is called **prompt engineering**.
- For instance, one can clearly specify the format of the desired answer in the prompt, give multiple choices, or even provide the model with one or more complete examples of what we are looking for (a demonstration). The latter approach is known as **few-shot learning**.
- The number of demonstrations doesn't need to be large; more examples seem to give diminishing returns, and can lead to overfitting. Demonstrations that have incorrect answers can still improve a system!
- Prompt engineering does not affect the weights of the model. In providing extra context, it **only affects the activation of the neurons**.

Prompt engineering

- Responses can also be optimized **after training**, by providing the model with contextual information about how the answer should look like. This is called **prompt engineering**.
- For instance, one can clearly specify the format of the desired answer in the prompt, give multiple choices, or even provide the model with one or more complete examples of what we are looking for (a demonstration). The latter approach is known as **few-shot learning**.
- The number of demonstrations doesn't need to be large; more examples seem to give diminishing returns, and can lead to overfitting. Demonstrations that have incorrect answers can still improve a system!
- Prompt engineering does not affect the weights of the model. In providing extra context, it **only affects the activation of the neurons**.

System prompts

- Chat-based LLMs like Clause or ChatGPT include hidden "**system prompts**" that act as preambles to any user query and "**set the tone**" of the request.
- The full system prompt for one language model Anthropic's Claude Opus4, is 1700 words long and includes sentences like the following:

> Claude should give **concise responses** to very simple questions, but provide **thorough responses** to complex and open-ended questions. Claude is able to explain difficult concepts or ideas **clearly**. It can also **illustrate** its explanations with examples, thought experiments, or metaphors. Claude **does not provide information that could be used to make chemical or biological or nuclear weapons** For more casual, emotional, empathetic, or advice-driven conversations, Claude keeps its tone natural, warm, and empathetic **Claude cares about people's well-being** and avoids encouraging or facilitating self-destructive behavior If Claude provides bullet points in its response, it should use markdown, and each bullet point should be at least 1-2 sentences long unless the human requests otherwise

- Of course, people find ways to "overwrite" these prompts, leading to sometimes dramatic misuses of the model.

System prompts

- Chat-based LLMs like Clause or ChatGPT include hidden "**system prompts**" that act as preambles to any user query and "**set the tone**" of the request.
- The full system prompt for one language model Anthropic's Claude Opus4, is 1700 words long and includes sentences like the following:

> Claude should give **concise responses** to very simple questions, but provide **thorough responses** to complex and open-ended questions. Claude is able to explain difficult concepts or ideas **clearly.** It can also **illustrate** its explanations with examples, thought experiments, or metaphors. Claude **does not provide information that could be used to make chemical or biological or nuclear weapons** For more casual, emotional, empathetic, or advice-driven conversations, Claude keeps its tone natural, warm, and empathetic **Claude cares about people's well-being** and avoids encouraging or facilitating self-destructive behavior If Claude provides bullet points in its response, it should use markdown, and each bullet point should be at least 1-2 sentences long unless the human requests otherwise

- Of course, people find ways to "overwrite" these prompts, leading to sometimes dramatic misuses of the model.

System prompts

- Chat-based LLMs like Clause or ChatGPT include hidden "**system prompts**" that act as preambles to any user query and "**set the tone**" of the request.
- The full system prompt for one language model Anthropic's Claude Opus4, is 1700 words long and includes sentences like the following:

> Claude should give **concise responses** to very simple questions, but provide **thorough responses** to complex and open-ended questions. Claude is able to explain difficult concepts or ideas **clearly.** It can also **illustrate** its explanations with examples, thought experiments, or metaphors. Claude **does not provide information that could be used to make chemical or biological or nuclear weapons** For more casual, emotional, empathetic, or advice-driven conversations, Claude keeps its tone natural, warm, and empathetic **Claude cares about people's well-being** and avoids encouraging or facilitating self-destructive behavior If Claude provides bullet points in its response, it should use markdown, and each bullet point should be at least 1-2 sentences long unless the human requests otherwise

- Of course, people find ways to "overwrite" these prompts, leading to sometimes dramatic misuses of the model.

Common issues with LLMs

Hallucinations

- LLMs can sometime answers or cite sources that look absolutely legit but are not real.
- This can have real consequences: fake court citations; fabricated academic citations (especially problematic in the domain of public health).
- A fix is Retrieval-augmented generation (**RAG**) is a technique that enables LLMs to retrieve and incorporate new information from databases, uploaded documents or web sources.
- Issue: what to do if those sources (the web especially) already contain LLMs hallucinations?

Biases

- Explicitly disclosing specific user information (**gender, ethnicity**...) can affect the model's responses in terms of offensiveness and general sentiment.

Biases

- Explicitly disclosing specific user information (**gender, ethnicity**...) can affect the model's responses in terms of offensiveness and general sentiment.
- Implicitly disclosing similar information, e.g. by using a **specific dialect**, also affects answers.

Biases

- Explicitly disclosing specific user information (**gender, ethnicity**...) can affect the model's responses in terms of offensiveness and general sentiment.
- Implicitly disclosing similar information, e.g. by using a **specific dialect**, also affects answers.
- Why is it the case, despite harm-mitigating system prompts?

Biases

- Explicitly disclosing specific user information (**gender, ethnicity**...) can affect the model's responses in terms of offensiveness and general sentiment.
- Implicitly disclosing similar information, e.g. by using a **specific dialect**, also affects answers.
- Why is it the case, despite harm-mitigating system prompts? Maybe because many biases are implicit, even in the training data. In other words, they surface in sources that otherwise exhibit no traces of toxicity.

Biases

- Explicitly disclosing specific user information (**gender, ethnicity**...) can affect the model's responses in terms of offensiveness and general sentiment.
- Implicitly disclosing similar information, e.g. by using a **specific dialect**, also affects answers.
- Why is it the case, despite harm-mitigating system prompts? Maybe because many biases are implicit, even in the training data. In other words, they surface in sources that otherwise exhibit no traces of toxicity.
- By replicating and sometimes amplifying biases, LLM can lead to more biases in the real world, for instance when being used to scan CVs, give professional/health-related advice. Also keep in mind that a lot of LLM "slop" gets reinjected back on the web...

Copyright issues (inspired by David's presentation to BA students)

- Anthropic's Claude was trained on more than **7 million pirated books**.
- Three authors sued Anthropic for **copyright infringement** by training on their works, and **illegal acquisition/storage** of pirated books by Anthropic.
- The former allegation was dismissed and classified as "**fair/transformative use**" (setting an important, though non-binding **precedent**). The latter allegation was upheld as copyright infringement.
- In September 2025, before the case went to full trial, Anthropic agreed to a $1.5 billion settlement and to delete the pirated datasets.
- Reminiscent of the Google Books case that happened around 20 years ago!!

Copyright issues (inspired by David's presentation to BA students)

- Anthropic's Claude was trained on more than **7 million pirated books**.
- Three authors sued Anthropic for **copyright infringement** by training on their works, and **illegal acquisition/storage** of pirated books by Anthropic.
- The former allegation was dismissed and classified as "**fair/transformative use**" (setting an important, though non-binding **precedent**). The latter allegation was upheld as copyright infringement.
- In September 2025, before the case went to full trial, Anthropic agreed to a $1.5 billion settlement and to delete the pirated datasets.
- Reminiscent of the Google Books case that happened around 20 years ago!!

Copyright issues (inspired by David's presentation to BA students)

- Anthropic's Claude was trained on more than **7 million pirated books**.
- Three authors sued Anthropic for **copyright infringement** by training on their works, and **illegal acquisition/storage** of pirated books by Anthropic.
- The former allegation was dismissed and classified as "**fair/transformative use**" (setting an important, though non-binding **precedent**). The latter allegation was upheld as copyright infringement.
- In September 2025, before the case went to full trial, Anthropic agreed to a $1.5 billion settlement and to delete the pirated datasets.
- Reminiscent of the Google Books case that happened around 20 years ago!!

Copyright issues (inspired by David's presentation to BA students)

- Anthropic's Claude was trained on more than **7 million pirated books**.
- Three authors sued Anthropic for **copyright infringement** by training on their works, and **illegal acquisition/storage** of pirated books by Anthropic.
- The former allegation was dismissed and classified as "**fair/transformative use**" (setting an important, though non-binding **precedent**). The latter allegation was upheld as copyright infringement.
- In September 2025, before the case went to full trial, Anthropic agreed to a $1.5 billion settlement and to delete the pirated datasets.
- Reminiscent of the Google Books case that happened around 20 years ago!!

Privacy issues

- Chat-based LLMs store an unsettling amount of information about past sessions and conversations. Just ask ChatGPT how much it knows about you!
- This poses the issue of how these sometimes personal and sensitive data are stored, who can access them, how to delete them etc.
- Another related issue is data leak from sensitive training data: with the right prompts **one can extract training-data text from a language model such as a person's name, phone number, and address**.

Privacy issues

- Chat-based LLMs store an unsettling amount of information about past sessions and conversations. Just ask ChatGPT how much it knows about you!
- This poses the issue of how these sometimes personal and sensitive data are stored, who can access them, how to delete them etc.
- Another related issue is data leak from sensitive training data: with the right prompts **one can extract training-data text from a language model such as a person's name, phone number, and address**.

Privacy issues

- Chat-based LLMs store an unsettling amount of information about past sessions and conversations. Just ask ChatGPT how much it knows about you!
- This poses the issue of how these sometimes personal and sensitive data are stored, who can access them, how to delete them etc.
- Another related issue is data leak from sensitive training data: with the right prompts **one can extract training-data text from a language model such as a person's name, phone number, and address**.

Changing how people speak or write (inspired by discussions with Sharon and Shelby)

- LLMs have been shown to repeatedly use distinctive “templates” or patterns: em dashes (–), sometimes combined with gerund clauses (e.g. *–deepening my understanding of the topic…*), parallel constructions like *it’s not only this, it’s that* or *it’s about this. It’s about that*; overall emphatic lexicon; no typos (!).
- Because everybody starts to be consciously or unconsciously aware of this, **people may try to adapt their usual style** to distinguish themselves from ChatGPT, which may sometimes lead them to **use suboptimal workarounds**!
- A consequence is that a **pragmatics of LLM use** might emerge: if I’m using a lot of “–” in my writing, then it might imply I did not even bother proof-reading ChatGPT’s output! Using previously neutral markers or constructions may now carry a negative message: I don’t take the task seriously.

Changing how people speak or write (inspired by discussions with Sharon and Shelby)

- LLMs have been shown to repeatedly use distinctive “templates” or patterns: em dashes (–), sometimes combined with gerund clauses (e.g. *–deepening my understanding of the topic…*), parallel constructions like *it’s not only this, it’s that* or *it’s about this. It’s about that*; overall emphatic lexicon; no typos (!).
- Because everybody starts to be consciously or unconsciously aware of this, **people may try to adapt their usual style** to distinguish themselves from ChatGPT, which may sometimes lead them to **use suboptimal workarounds**!
- A consequence is that a **pragmatics of LLM use** might emerge: if I’m using a lot of “–” in my writing, then it might imply I did not even bother proof-reading ChatGPT’s output! Using previously neutral markers or constructions may now carry a negative message: I don’t take the task seriously.

Changing how people speak or write (inspired by discussions with Sharon and Shelby)

- LLMs have been shown to repeatedly use distinctive “templates” or patterns: em dashes (–), sometimes combined with gerund clauses (e.g. *–deepening my understanding of the topic…*), parallel constructions like *it’s not only this, it’s that* or *it’s about this. It’s about that*; overall emphatic lexicon; no typos (!).
- Because everybody starts to be consciously or unconsciously aware of this, **people may try to adapt their usual style** to distinguish themselves from ChatGPT, which may sometimes lead them to **use suboptimal workarounds**!
- A consequence is that a **pragmatics of LLM use** might emerge: if I’m using a lot of “–” in my writing, then it might imply I did not even bother proof-reading ChatGPT’s output! Using previously neutral markers or constructions may now carry a negative message: I don’t take the task seriously.

Lab

https://colab.research.google.com/drive/1ZV0wlm0xsRO-QAQyVE4RbsL31jvfT16O0?usp=sharing